

Kazakhstan Practical Driving Exam Simulator

Category B - Astana Autodrome

Damir Tassybayev
tassybayev.kostanay@gmail.com

Course: Interactive Graphics
Instructor: Prof. Marco Schaerf
Sapienza University of Rome

Contents

1	Introduction	2
1.1	Graphics environment	2
1.2	Running the simulator	2
1.3	Source layout	2
2	User Manual	2
2.1	Premise and objective	2
2.2	Controls	3
2.3	Camera modes	3
2.4	HUD overview	3
2.5	Examination flow and exercises	4
3	The Car: hierarchical model	4
3.1	Part hierarchy and assembly	4
3.2	Materials and lights	5
3.3	Constructing the parts	5
4	The Autodrome (environment)	6
4.1	Map image and pixel-to-world mapping	6
4.2	The overpass (estakada)	6
4.3	Course markings and traffic lights	6
4.4	Lighting and shadows	7
4.5	Procedural asphalt textures	7
5	Driving model	7
5.1	Kinematic bicycle model	7
5.2	Longitudinal dynamics	8
5.3	Steering response	8
5.4	Riding the terrain	8
5.5	Handbrake and stall	8
5.6	Keyboard input	9
6	Animations	9
6.1	Doors via tween.js	9
6.2	Turn indicators and lights	10
7	The examiner: scoring system	10
7.1	Scoring engine	10
7.2	Data-driven course	10
7.3	Line-crossing detection	10
7.4	Parking and hill exercises	11
7.5	The course as data	11
8	Rendering loop and orchestration	11
9	Performance and design choices	12
10	Course requirements coverage	13
11	Limitations and future work	13
12	Resource attribution	13

1 Introduction

This project is a browser-based 3D simulation of the **practical driving examination of the Republic of Kazakhstan** for the category-B licence. The player drives a car around a faithful recreation of an Astana *autodrome* (driving-test ground) and is graded, in real time, by an automated examiner that reproduces the official penalty system: turn-signal checkpoints, a hill start (*estakada*), traffic lights, boundary lines, and two timed parking exercises. The simulator was developed as the final project for the Interactive Graphics course at Sapienza University of Rome.

1.1 Graphics environment

The application runs in the browser through WebGL, using **Three.js** (release r160) as the rendering layer and **tween.js** for eased interpolation. The renderer is a **WebGLRenderer** with antialiasing enabled, the **sRGB** output colour space, soft shadows (**PCFSoftShadowMap**), and a device pixel ratio capped at 2 to avoid over-drawing on high-DPI screens.

A deliberate design choice is that the project has **no build step**. All modules are native ES modules resolved through an **importmap** in **index.html**, and every third-party library is vendored locally under **lib/**. This keeps the repository self-contained and lets it run directly from GitHub Pages (or any static file server) without bundling.

1.2 Running the simulator

Because the project uses ES modules, it must be served over HTTP rather than opened from the file system. Any static server works, for example:

```
1 python -m http.server 8000      # from the repository root
2 # then open http://localhost:8000
```

Listing 1: Running the simulator locally.

It has been tested on Chromium-based browsers and Firefox on Windows. Because the controls are keyboard-driven, mobile devices are not supported.

1.3 Source layout

The codebase is organised as small single-responsibility ES modules under **src/**, summarised in Table 1. The world is assembled in **main.js**, which owns the render loop and wires the pieces together; the simulation logic (driving and examination) is kept separate from the rendering so each can be reasoned about on its own.

2 User Manual

2.1 Premise and objective

The candidate starts the car at the START box at the top of the autodrome and drives a fixed route through a sequence of exercises. The automated examiner accumulates *penalty points* for mistakes. The examination is **passed** if, and only if, the candidate completes the course and the cumulative penalty stays *below 100 points*; reaching 100 points, whether through one critical error or several smaller ones, ends the examination immediately as a failure. Penalties fall into three severity tiers, summarised in Table 2.

Table 1: Source modules and their responsibilities.

Module	Responsibility
<code>index.html</code>	Entry point, import map, HUD markup, styles
<code>main.js</code>	Bootstraps the renderer/scene, owns the main loop and HUD
<code>input.js</code>	Keyboard state: held keys and one-shot “tap” handlers
<code>cameras.js</code>	Camera manager: fixed / cockpit / top / orbit views
<code>vehicle.js</code>	Kinematic driving model and terrain ride
<code>world/car.js</code>	Hierarchical car model and its animations
<code>world/environment.js</code>	Sky, fog, sun and hemisphere lights, surrounding ground
<code>world/examGround.js</code>	The autodrome image laid on the ground plane
<code>world/autodromeMap.js</code>	World/image dimensions and the car start pose
<code>world/mapCoords.js</code>	Image-pixel to world-coordinate conversion
<code>world/overpass.js</code>	The hill-start overpass (geometry + height field)
<code>world/courseMarks.js</code>	Boundary lines and the traffic-light models
<code>world/trafficLight.js</code>	Traffic-light model and signal cycle
<code>world/textures.js</code>	Procedural asphalt colour/normal/roughness maps
<code>exam/scoring.js</code>	Penalty accumulation and pass/fail logic
<code>exam/course.js</code>	The examination route as data, in image pixels
<code>exam/exam.js</code>	Generic examiner engine evaluating the course
<code>lib/</code>	Vendored Three.js (+ addons) and tween.js

Table 2: Penalty severity tiers.

Points	Severity	Examples
100	Critical (instant fail)	Crossing on a red/yellow light; not finishing a parking in time
25	Major violation	Missing a full stop; heavy hill rollback
5	Minor mistake	Missing a turn signal at a checkpoint
20	Boundary touch	A wheel on a penalty (boundary) line

2.2 Controls

The car is driven with the keyboard; the mouse orbits the camera only in the free-orbit view. The full scheme is listed in Table 3.

2.3 Camera modes

The **C** key cycles four views, handled by the **CameraManager**. The default *fixed* view is rigidly bolted to the car: it translates and rotates with the body so the car always sits in the same place on screen. *Cockpit* places the camera at the driver’s seat looking forward. *Top* looks straight down, useful for the parking exercises. *Orbit* frees the camera with **OrbitControls** so the player can drag and zoom around the car. The non-orbit modes are positioned each frame from a car-local offset converted to world space, so they follow the body exactly.

2.4 HUD overview

Two panels overlay the scene. The top-left panel reports the live vehicle state: speed in km/h, gear (D, R or P when the handbrake is engaged), the active camera, the headlight state, and the current turn signal. The top-right *examiner* panel shows the running penalty total, colour-coded from green to amber to red as it climbs, together with the most recent violation. When the examination ends, a full-screen overlay announces PASSED or FAILED with the final penalty total and, on failure, the reason.

Table 3: Controls.

Input	Action
W / S or $\uparrow$ / $\downarrow$	Accelerate / brake and reverse
A / D or $\leftarrow$ / $\rightarrow$	Steer left / right
Space	Full-stop handbrake (toggle)
Q / E	Left / right turn signal (toggle)
H	Hazard lights (toggle)
C	Cycle the camera view
L	Toggle the headlights
O	Open / close the doors
R	Reset the car and the examination
Mouse drag / wheel	Orbit / zoom (in the <i>orbit</i> camera only)

2.5 Examination flow and exercises

The simulator follows the structure of the real Kazakh practical test, which is organised in three phases. *Phase 1 (preparation and start)* begins at the START box; the candidate must leave with the correct turn signal active before crossing the start line. *Phase 2 (circuit exercises)* is the body of the test, a fixed sequence of manoeuvres each with strict boundaries:

1. **Hill start (estakada).** Drive up the ramp, stop at the STOP line, and pull away without rolling back.
2. **Controlled intersections.** City-style crossroads governed by traffic lights; the candidate must not enter on a red or yellow signal.
3. **90° turns.** A narrow corridor of right angles bounded by lines that must not be touched.
4. **Slalom and route checkpoints.** A series of turns, each requiring the appropriate turn signal.
5. **90° reverse (box) parking.** Reverse the car into a marked garage box within the time limit.
6. **Parallel parking.** Reverse into a kerbside bay within the time limit.

Phase 3 (general rules) applies throughout: signalling for every turn, not crossing on a restrictive light, and observing the full-stop requirements. The run ends when the car crosses the finish line, at which point the verdict is shown. Every one of these checks is expressed as data and evaluated by the generic engine described in Section 7; the exact positions and penalties are listed in Table 4.

3 The Car: hierarchical model

The car is the project's complex hierarchical model and is built entirely in code in `world/car.js`; nothing about it is imported, in line with the course rule that animations may not be imported.

3.1 Part hierarchy and assembly

The car is a tree of `THREE.Group` nodes. The root node carries the whole car and is the only node the driving model moves; every child therefore follows automatically. The hierarchy is:

- **root** — position and heading set by the driving model;

- **body** — lower body, cabin, glass band, chrome bumpers, seats; the emissive head-, brake- and indicator-light lenses; the two hinged *doors*; and the *steering wheel* (a torus nested in two helper groups so it can spin about its own axle regardless of its tilt);
- **front-left** and **front-right pivots** — each a group that yaws for steering and contains a wheel that spins for rolling;
- **rear-left** and **rear-right wheels** — spin only;
- **two headlight spot lights** aimed forward.

The car's local forward direction is $+X$ and up is $+Y$. Key dimensions are a half-wheelbase of 1.45 m, a half-track of 0.82 m, a wheel radius of 0.42 m and a steering lock of `MAX_STEER` = 0.52 rad ($\approx 30^\circ$). Because the front wheels are children of their steering pivots, they turn in place and still roll correctly; this two-level joint/visual split is what lets every joint rotate independently without disturbing the others.

3.2 Materials and lights

The body uses `MeshStandardMaterial` instances (metallic paint, dark glass, chrome bumpers, rubber tyres). The light lenses use emissive materials whose intensity is animated: the headlight lenses and two non-shadowing `SpotLights` fade in when the lights are toggled, the brake lenses brighten while braking or reversing, and the amber indicator lenses blink at about 2 Hz. Car-local *left* is the $-Z$ side and *right* is $+Z$, so the left/right indicators light the correct side of the body.

3.3 Constructing the parts

Each wheel is a small group of its own: a cylinder rotated so its axle lies along the local Z axis, plus a hub cap and a spoke so the spin is visible. Building the axle this way means rolling is a rotation about a single local axis, and the front wheels can be parented under a steering pivot without any further bookkeeping.

```

1 function makeWheel(tireMat, hubMat) {
2   const g = new THREE.Group();
3   const tireGeo = new THREE.CylinderGeometry(WHEEL_RADIUS, WHEEL_RADIUS,
4     WHEEL_WIDTH, 22);
5   tireGeo.rotateX(Math.PI / 2); // axle along local Z
6   g.add(new THREE.Mesh(tireGeo, tireMat));
7   const hubGeo = new THREE.CylinderGeometry(0.16, 0.16, WHEEL_WIDTH + 0.02, 12);
8   hubGeo.rotateX(Math.PI / 2);
9   g.add(new THREE.Mesh(hubGeo, hubMat)); // hub cap
10  g.add(new THREE.Mesh(new THREE.BoxGeometry(0.62, 0.06, WHEEL_WIDTH + 0.03),
11    hubMat));
12  return g;
13 }
```

Listing 2: Wheel construction (`car.js`).

The doors are the clearest use of the hierarchy: each door is a hinge group whose origin sits at the front edge of the door, with the panel hanging behind it. Opening is then a single yaw of the hinge group, mirrored between the two sides. The steering wheel is mounted in two nested helper groups (a yaw to face the driver and a tilt for the column), so that spinning it is always a rotation about its own axle no matter how the column is angled. The car's pose is written each frame as a quaternion so the body can be both yawed (heading) and pitched (on the ramp) at once:

```

1 setPose(yaw, pitch = 0) {
2   _qy.setFromAxisAngle(_Y, yaw);
3   _qp.setFromAxisAngle(_Z, pitch);
4   root.quaternion.copy(_qy).multiply(_qp);
5 }

```

Listing 3: Combined yaw and pitch as a quaternion (`car.js`).

4 The Autodrome (environment)

4.1 Map image and pixel-to-world mapping

The driving ground is a single photographic image of a real Astana autodrome, laid flat on a textured plane (`world/examGround.js`). The image (1237×848 px) is loaded asynchronously with `TextureLoader` and the ground plane is sized to the *same aspect ratio*, so a pixel on the image maps exactly to a point on the ground. The plane is oriented so that the image top is world north ($-Z$) and the image right is world east ($+X$).

Every element of the examination is positioned by reading its location directly off the image in pixels and converting it with the helper in `world/mapCoords.js`:

```

1 export function px2world(px, py) {
2   return {
3     x: (px / IMG_W - 0.5) * AUTO_W,    // 0..IMG_W (left->right)  -> world X
4     z: (py / IMG_H - 0.5) * AUTO_H,    // 0..IMG_H (top->bottom)  -> world Z
5   };
6 }

```

Listing 4: Pixel-to-world mapping (`mapCoords.js`).

This single source of truth keeps "what is painted on the image" and "what the car interacts with" in agreement: the car start, the hill, and all examination lines are expressed in image pixels and converted through `px2world`.

4.2 The overpass (estakada)

The hill-start exercise is a real raised structure built in `world/overpass.js`. Its footprint is taken from image pixels (it spans image $x = 550$ on the east, where the west-bound car arrives, to $x = 210$ on the west). The deck is an extruded side profile: an up-ramp, a flat crest carrying a painted STOP line and a STOP sign on the right edge, and a down-ramp. A height function returns the deck elevation at any point, and zero elsewhere:

```

1 function heightAt(x, z) {
2   if (z < z0 || z > z1 || x > xA || x < xD) return 0;
3   if (x >= xB) return H * (xA - x) / (xA - xB); // up ramp
4   if (x <= xC) return H * (x - xD) / (xC - xD); // down ramp
5   return H;                                     // crest
6 }

```

Listing 5: Overpass deck height (`overpass.js`).

The driving model samples this function so the car physically rides up, over and down the deck (Section 5).

4.3 Course markings and traffic lights

The examination lines that must be visible are drawn on the ground by `world/courseMarks.js`: the boundary "penalty" lines as thin black strips, and a red stop line at each traffic-light location.

At each traffic light a small hierarchical light model (post, housing and three lamps) is placed and driven by a green $\rightarrow$ yellow $\rightarrow$ red cycle, desynchronised between lights. The remaining rule lines (turn-signal, full-stop, parking, finish) are invisible triggers by default to keep the photographic map clean; a debug flag draws them all for verification.

4.4 Lighting and shadows

The scene is lit by a `HemisphereLight` ambient term, so no surface is fully black, and a `DirectionalLight` acting as the sun, which casts the scene shadows through a 2048×2048 shadow map. The car's headlights add two forward spot lights at night. Shadows use percentage-closer filtering for soft edges, and the renderer works in the sRGB colour space for correct tone.

4.5 Procedural asphalt textures

The ground that surrounds the photographic map is a tiling asphalt surface whose three maps are generated at run time on an HTML canvas, with no external image files (`world/textures.js`). This satisfies the requirement for textures of *different kinds*: a *colour* map (dark grey with per-pixel noise and occasional light speckles), a *normal* map (a per-pixel perturbation of the X/Y normal channels for micro-surface detail under the directional light), and a *roughness* map (patchy grey so reflections vary across the surface). All three tile and use anisotropic filtering. The normal map is the most illustrative:

```

1  const normal = canvas(size, (ctx) => {
2    const img = ctx.createImageData(size, size);
3    for (let i = 0; i < img.data.length; i += 4) {
4      img.data[i]      = 128 + (Math.random() - 0.5) * 70; // X
5      img.data[i + 1] = 128 + (Math.random() - 0.5) * 70; // Y
6      img.data[i + 2] = 255;                               // Z (mostly up)
7      img.data[i + 3] = 255;
8    }
9    ctx.putImageData(img, 0, 0);
10 });

```

Listing 6: Procedural normal map (`textures.js`).

5 Driving model

5.1 Kinematic bicycle model

The car is driven by a kinematic *bicycle model* (`vehicle.js`); no physics engine is used, keeping the motion light and predictable. The state is the planar position (x, z) , the heading ψ , the signed speed v , and the current steering angle δ . Throttle accelerates, braking decelerates and then engages reverse, and releasing both applies rolling friction toward zero. With wheelbase $L = 2.9$ m the pose integrates as

$$\dot{\psi} = \frac{v}{L} \tan \delta, \quad \dot{x} = v \cos \psi, \quad \dot{z} = -v \sin \psi.$$

Speed is clamped to $[-5, 15]$ m/s (≈ -18 to 54 km/h) and the steering lock is reduced at higher speed for stability. The travelled distance is fed back to the wheels so the rolling animation always matches the real speed.

```

1  if (Math.abs(this.speed) > 1e-3) {
2    this.heading += (this.speed / WHEELBASE) * Math.tan(this.steer) * dt;
3  }
4  this._dir.set(Math.cos(this.heading), 0, -Math.sin(this.heading));

```



```

5  const dist = this.speed * dt;
6  this.x += this._dir.x * dist;
7  this.z += this._dir.z * dist;

```

Listing 7: Pose integration (`vehicle.js`).

5.2 Longitudinal dynamics

Speed is integrated from the throttle input. Holding forward accelerates at 9 m/s^2 ; holding back brakes at 20 m/s^2 until the car stops and then engages reverse at a reduced rate; releasing both applies a rolling friction of 3.2 m/s^2 that pulls the speed toward zero. The result is clamped to the allowed range.

```

1  if (throttle > 0) {
2    this.speed += ENGINE_ACCEL * dt;           // accelerate
3  } else if (throttle < 0) {
4    if (this.speed > 0.1) this.speed -= BRAKE_DECEL * dt;    // brake
5    else                 this.speed -= ENGINE_ACCEL * 0.6 * dt; // reverse
6  } else {
7    const f = ROLL_FRICTION * dt;
8    this.speed += this.speed > 0 ? -Math.min(f, this.speed)
9                  : Math.min(f, -this.speed);
10 }
11 this.speed = THREE.MathUtils.clamp(this.speed, MAX_REV, MAX_FWD);

```

Listing 8: Longitudinal model (`vehicle.js`).

5.3 Steering response

The steering angle is not applied directly from the key state; instead a target angle is computed and the current angle eases toward it, which gives the wheels a smooth, momentum-like feel. The steering lock is also scaled down with speed so the car is calm at speed and tight when manoeuvring:

```

1  const speedFactor = 1 - 0.45 * Math.min(1, Math.abs(this.speed) / MAX_FWD);
2  const targetSteer = steerIn * MAX_STEER * speedFactor;
3  this.steer += (targetSteer - this.steer) * Math.min(1, STEER_SPEED * dt);

```

Listing 9: Eased, speed-sensitive steering (`vehicle.js`).

5.4 Riding the terrain

When the car is over the overpass, the model raises it to the deck height and pitches it to the local slope. The pitch is taken from two height samples a short distance e ahead of and behind the car along its travel direction, and a component of gravity along the slope is added to the speed, so the hill must be actively held:

$$\theta_{\text{pitch}} = \text{atan2}(h_f - h_b, 2e), \quad \dot{v} += -g \sin \theta_{\text{pitch}}.$$

The resulting position and orientation are pushed to the car root as a quaternion (yaw then pitch), so the body sits correctly on the ramp.

5.5 Handbrake and stall

Pressing `Space` toggles a full-stop handbrake: the speed is forced to zero and throttle is ignored until it is released, with the brake lights held on. The model also detects a stall when throttle and brake are held together near a standstill, which is reported to the examiner.

5.6 Keyboard input

Input is handled by a small `Keyboard` class (`input.js`) that distinguishes two kinds of key. Continuous actions, such as accelerating and steering, query whether a key is currently held; one-shot actions, such as toggling a light or cycling the camera, register a callback that fires once per fresh key-press. This keeps the main loop simple: it reads the held state for driving and lets the tap handlers deal with toggles. Arrow keys and the space bar have their default page-scrolling behaviour suppressed.

```
1 down(...codes) { return codes.some((c) => this.held.has(c)); } // continuous
2 tap(code, callback) { this.onTap.set(code, callback); } // once per press
```

Listing 10: Held vs. one-shot keys (`input.js`).

6 Animations

All animation is computed in JavaScript every frame; nothing is imported.

- **Wheels.** All four wheels spin about their axle by an angle that advances with the travelled distance, $\Delta\theta = \frac{ds}{r}$, where r is the wheel radius. The two front wheels additionally yaw with the steering pivots.
- **Steering wheel.** The cabin steering wheel rotates about its own axle proportionally to the steering angle.
- **Doors.** Each door is a hinge group; opening and closing is driven by `tween.js` with a `Quadratic.Out` easing for a smooth swing.
- **Lights.** Headlight and brake intensities ease toward their target; the turn indicators blink; the brake lenses respond to braking and reversing.
- **Traffic lights.** Each light advances through its green/yellow/red cycle and updates the emissive lamp intensities.

6.1 Doors via `tween.js`

The door swing is the one place where the project uses `tween.js`, as suggested in the course material. Toggling the doors starts a tween on a shared group that eases an `open` parameter between 0 and 1 with a `Quadratic.Out` curve; an `onUpdate` callback applies the (mirrored) hinge rotations. The tween group is advanced once per frame from the main loop.

```
1 toggleDoors() {
2   doorTarget = doorTarget > 0.5 ? 0 : 1;
3   new TWEEN.Tween(doorState, tweens)
4     .to({ open: doorTarget }, 650)
5     .easing(TWEEN.Easing.Quadratic.Out)
6     .onUpdate(applyDoors) // doorL.rotation.y = open; doorR = -open
7     .start();
8 }
```

Listing 11: Eased door swing (`car.js`).

6.2 Turn indicators and lights

The amber indicators are driven by a square wave on an accumulated timer, so the correct side (or both, for hazards) blinks at roughly 2Hz. The headlight and brake intensities are eased toward their targets rather than switched, which reads like a bulb warming up.

```

1 blinkT += dt;
2 const lit = indicator !== 'off' && (blinkT % 0.5) < 0.25 ? 1.5 : 0.0;
3 indLeftMat.emissiveIntensity = (indicator === 'left' || indicator === 'hazard'
4   ) ? lit : 0;
5 indRightMat.emissiveIntensity = (indicator === 'right' || indicator === 'hazard'
6   ) ? lit : 0;

```

Listing 12: Indicator blink (`car.js`).

7 The examiner: scoring system

The examination logic is a generic, data-driven engine that reads a course definition and applies a rule per element each frame, accumulating penalties.

7.1 Scoring engine

The `Scoring` class (`exam/scoring.js`) keeps a running total and an ordered event log, and fails the examination as soon as a critical (100-point) penalty fires or the total reaches the threshold:

```

1 penalize(exercise, rule, points, t = 0) {
2   if (this.isOver) return;
3   this.total += points;
4   this.log.push({ t, exercise, rule, points });
5   this.lastViolation = this.log[this.log.length - 1];
6   if (points >= FAIL_THRESHOLD || this.total >= FAIL_THRESHOLD) {
7     this.status = 'failed';
8     this.failReason = rule;
9   }
10 }

```

Listing 13: Penalty accumulation (`scoring.js`).

7.2 Data-driven course

The whole route lives in one editable configuration file, `exam/course.js`, with every position expressed in image pixels. It groups the elements by type: *signal lines* (a crossing requires a given indicator), *light lines* (a crossing on red/yellow fails), a *stop line* (a full stop is required before crossing), *penalty lines* (a wheel on the line costs points), two timed *parkings*, and the *finish line*. The engine (`exam/exam.js`) converts each pixel segment to a world segment once at start-up and then evaluates it generically.

7.3 Line-crossing detection

The car is reduced to a reference point (its front axle) and its four wheels in world space. A line is the segment $A \rightarrow B$; the side of the car relative to the line is the sign of the 2-D cross product

$$s = (B_x - A_x)(P_z - A_z) - (B_z - A_z)(P_x - A_x).$$

A crossing is registered when s flips sign between frames *and* the projection of P onto the segment lies within its extent. Each rule then reacts: a signal line checks the active indicator,

a light line reads its traffic signal, the stop line checks that a one-second stop was registered nearby, and the finish line ends the examination. The penalty (boundary) lines instead use a point-to-segment distance against each wheel, with a short cooldown so a wheel resting on a line is charged once rather than every frame.

```

1  _cross(key, px, pz, L) {
2      const s = sideOf(px, pz, L);           // sign of the 2-D cross product
3      const prev = this._side.get(key);
4      this._side.set(key, s);
5      if (prev === undefined || prev === 0 || s === 0 || prev === s) return false;
6      const t = projT(px, pz, L);           // projection onto the segment
7      return t >= -0.15 && t <= 1.15;       // crossed within the segment's extent
8  }

```

Listing 14: Segment-crossing test (`exam.js`).

7.4 Parking and hill exercises

Each parking exercise arms when the car crosses its trigger line and then gives a fixed time budget (30 s). It succeeds when the required wheels (the two rear wheels for the 90° box, the two right wheels for parallel parking) are held stationary inside the target line for one second; if the budget expires first, the examination fails. The hill exercise is scored against the overpass footprint: rolling back more than 30 cm from the furthest-forward point, or stopping more than 0.5 m from the STOP line (or not stopping at all), each costs a major penalty.

```

1  if (!ps.active && !ps.done) {
2      if (this._cross('park:' + p.id, px, pz, p.trig)) { ps.active = true; ps.t = 0;
3      }
4  } else if (ps.active) {
5      ps.t += dt;
6      const need = p.wheels === 'rear' ? ['rl', 'rr'] : ['fr', 'rr'];
7      const inTarget = need.every((k) => distToSeg(W[k].x, W[k].z, p.targ) < 1.4);
8      if (inTarget && Math.abs(v.speed) < 0.2) { ps.hold += dt; if (ps.hold >= 1) ps
9          .done = true; }
10     else ps.hold = 0;
11     if (ps.active && ps.t > p.limit) sc.penalize('Parking', `${p.name} not in time
12         ', 100);
13 }

```

Listing 15: Timed parking evaluation (`exam.js`).

7.5 The course as data

Table 4 lists the complete examination route as it appears in `course.js`. Every element is a line segment given in *image pixels* (the 1237×848 map), which the engine converts to world coordinates at start-up. Because the whole route is data, moving a checkpoint or changing a penalty is a one-line edit, and the rendering and the scoring can never disagree about where a line is.

8 Rendering loop and orchestration

`main.js` assembles the world once and then runs a fixed per-frame order in `animate`. The frame time is read from a clock and capped at 50 ms so a momentary stall cannot teleport the car. Input is read, the vehicle and car are advanced, the traffic lights and the examiner are updated, the tween group is stepped, the camera is positioned, the HUD is refreshed, and finally the scene is drawn.

Table 4: The examination course (positions in image pixels).

Element	Rule	A (px)	B (px)	Points
<i>Turn-signal checkpoints</i>				
start	left signal	(820, 30)	(820, 62)	5
s5	left signal	(40, 285)	(75, 285)	5
s7	right signal	(550, 230)	(550, 265)	5
s10	right signal	(1000, 430)	(1000, 460)	5
s12	right signal	(1103, 457)	(1103, 485)	5
s13	right signal	(685, 550)	(685, 800)	5
s15	right signal	(627, 280)	(672, 280)	5
s17	right signal	(1081, 243)	(1081, 264)	5
s19	left signal	(116, 409)	(116, 440)	5
s21	left signal	(43, 773)	(72, 773)	5
<i>Traffic-light stop lines (fail on red/yellow)</i>				
l8	traffic light	(590, 350)	(625, 350)	100
l9	traffic light	(543, 433)	(542, 464)	100
l14	traffic light	(623, 516)	(651, 516)	100
<i>Full-stop line</i>				
st20	full stop	(42, 663)	(71, 663)	25
<i>Boundary (penalty) lines</i>				
b1–b6	wheel on line	6 segments near (290–380, 240–385)		20
<i>Timed parking (fail if not parked in 30 s)</i>				
box (trigger)	enter	(1030, 530)	(1066, 530)	100
box (target)	rear wheels	(1102, 655)	(1102, 687)	—
parallel (trigger)	enter	(627, 280)	(672, 280)	100
parallel (target)	right wheels	(706, 302)	(853, 300)	—
finish	ends the run	(1081, 243)	(1081, 264)	—

```

1 function animate() {
2   requestAnimationFrame(animate);
3   const dt = Math.min(clock.getDelta(), 0.05); // cap the step
4   // read held keys -> throttle, steerIn, both-pedals
5   vehicle.update(dt, throttle, steerIn, up && dn);
6   car.update(dt);
7   courseMarks.update(dt); // advance the traffic-light cycles
8   exam.update(dt); // evaluate the course, accumulate penalties
9   tweens.update(performance.now()); // door easing
10  cameras.update();
11  updateHud();
12  renderer.render(scene, camera);
13 }

```

Listing 16: Per-frame order (`main.js`).

9 Performance and design choices

A few deliberate choices keep the frame rate steady and the project simple to deploy. There is **no build step**: native ES modules and an import map mean the repository runs as-is on GitHub Pages, and every library is vendored so nothing has to be fetched at run time. Only *one* light casts shadows (the sun); the headlights are shadowless. The device pixel ratio is capped at 2 so high-density displays are not forced to draw far more pixels than needed, and the frame step is capped as shown above. The map image is loaded asynchronously, with a neutral grey ground shown until it arrives, and the surrounding asphalt uses procedural canvas textures rather than downloaded image files. All geometry is low-poly primitives, so the scene runs comfortably on

integrated graphics.

10 Course requirements coverage

Table 5 maps the four mandatory requirements of the course project to where they are met in the simulator.

Table 5: Coverage of the mandatory project requirements.

Requirement	Where it is satisfied
Hierarchical model	The category-B car (Section 3): a multi-level scene graph whose wheels, steering wheel and doors are animated through its structure.
Lights and textures	Hemisphere ambient, a shadow-casting sun and two headlight spot lights; procedural asphalt colour, normal and roughness maps, plus the photographic ground texture.
User interaction	Keyboard driving, four camera modes, turn signals, hazards, headlights, doors, a full-stop handbrake, and reset.
Animations (hand-written)	Wheels, steering wheel, doors (tween.js), light fades and blinks, traffic-light cycles and the hill ride — all computed in JavaScript, with nothing imported.

11 Limitations and future work

The simulation is intentionally kinematic, so it does not model tyre slip, weight transfer or collisions between the car and the scenery. The route assumes the candidate follows the intended path; the turn-signal direction expected at each checkpoint is fixed in the course data rather than inferred from the manoeuvre. The traffic-light orientations and the parking tolerances were tuned by hand and could be refined. Natural extensions include audio feedback, a guided-route overlay on the HUD, a scored debrief listing every penalty event, and an optional lightweight physics layer for cone and kerb collisions.

12 Resource attribution

Libraries (not developed by the team).

- **Three.js** (MIT) — <https://threejs.org/>; release r160, used for rendering, the scene graph and the `OrbitControls` addon.
- **tween.js** (MIT) — <https://github.com/tweenjs/tween.js/>; eased interpolation for the door animation.

Both libraries are vendored under `lib/` so the repository is self-contained.

Assets. The autodrome ground is a single photographic top-down image of an Astana driving-test ground (`assets/textures/autodrome.png`). All other geometry (the car, the overpass, the traffic lights, the course markings) and all animation are generated in code by the team; no 3D models or animation clips are imported.

Code. All application code under `src/` was written by the team. The procedural asphalt textures are generated at run time on an HTML canvas.